

Waymo Sensor Fusion Processor



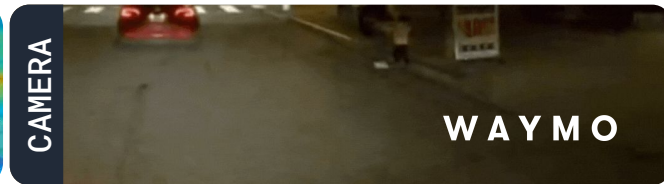
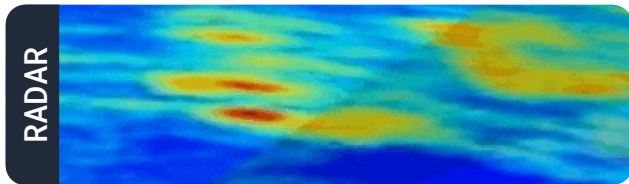
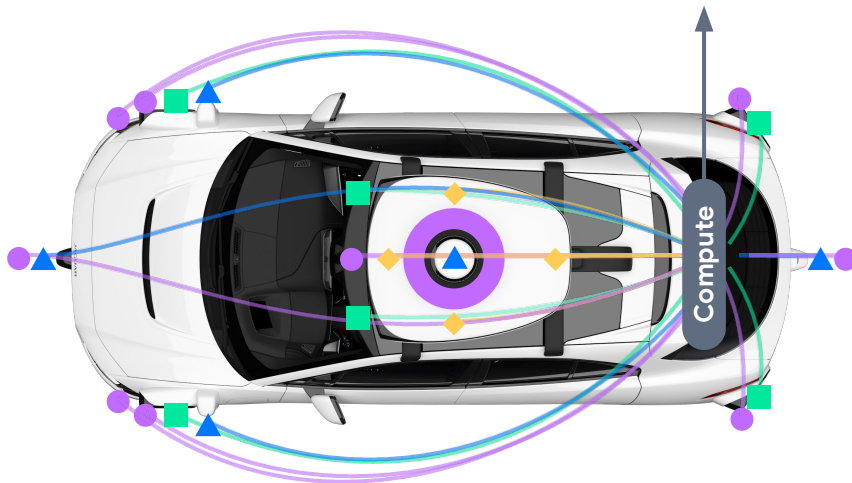
WAYMO



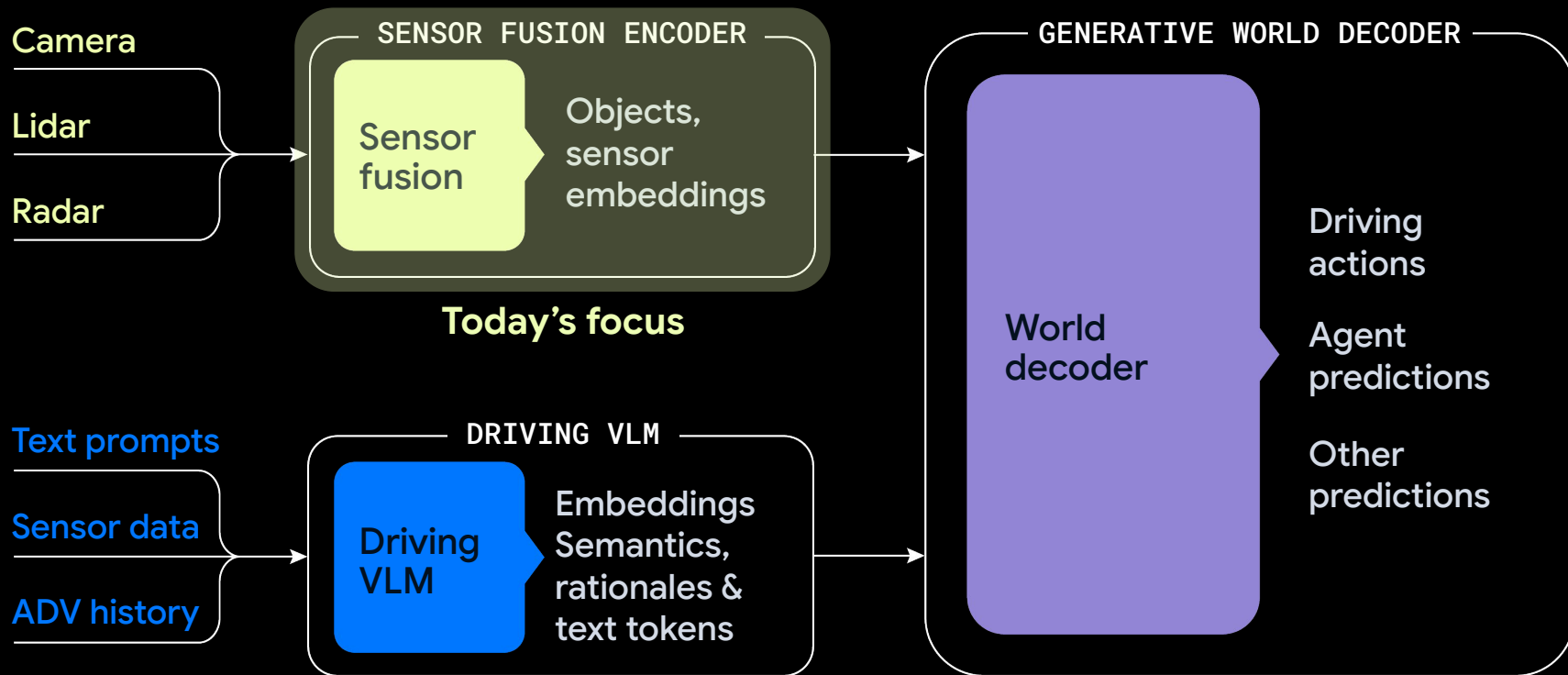
Compute in the car

- ▲ Lidar system
- Vision system
- Radar system
- ◆ Microphone

Compute is at the center of the system

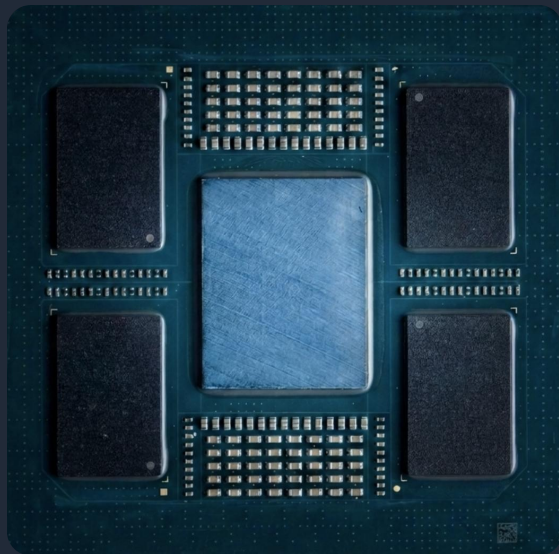


The Waymo Foundation Model



ADV = Autonomous Driving Vehicle
VLM = Vision Language Model

Design goals



Vertical integration

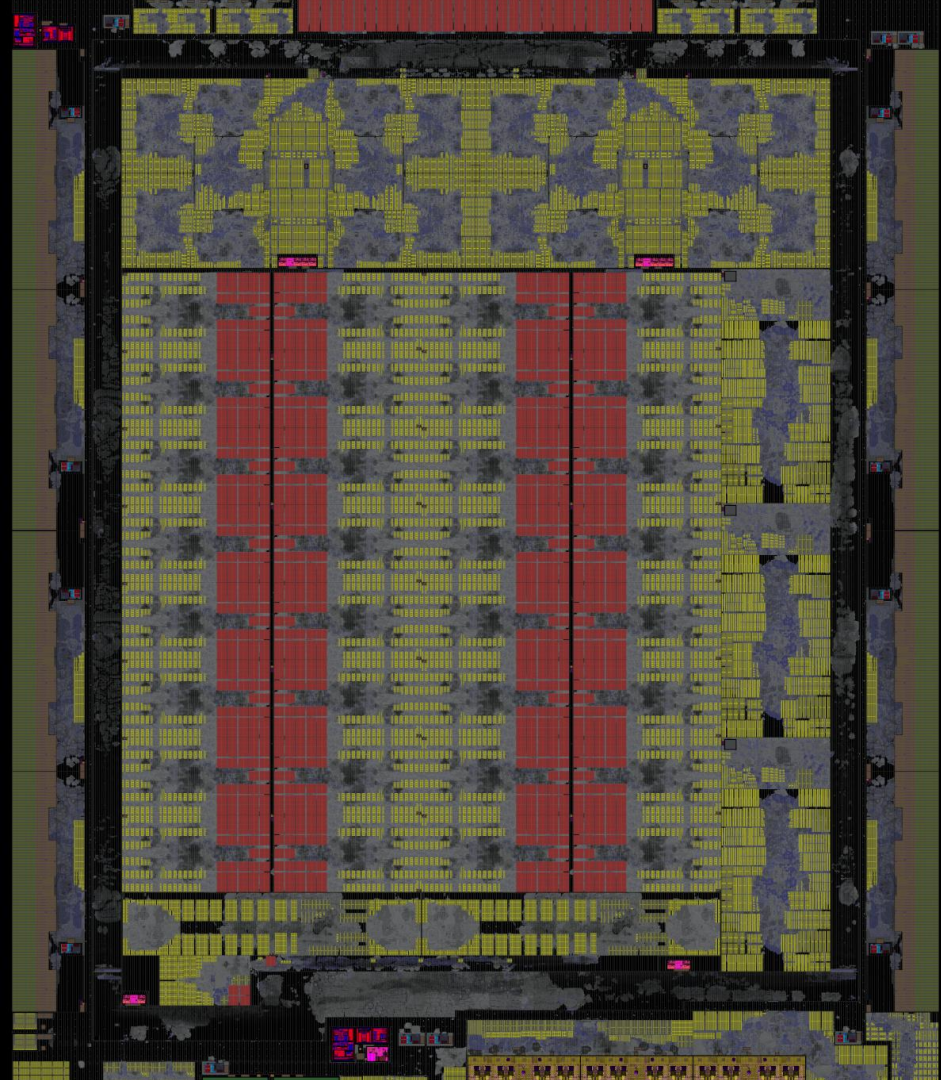
- Waymo-specific (in-house) sensor processing
 - Co-designed for Waymo sensor fusion ML models
-

Optimized for latency

- Low batch
 - First pixel in to embedding out
-

Engineered for autonomy at scale

- Reliability and operating conditions
- Safety, security



Key specifications

Area

208 mm² in N5

Package

45 mm x 45 mm FCBGA

Memory technology

In-package LPDDR5x

Memory BW

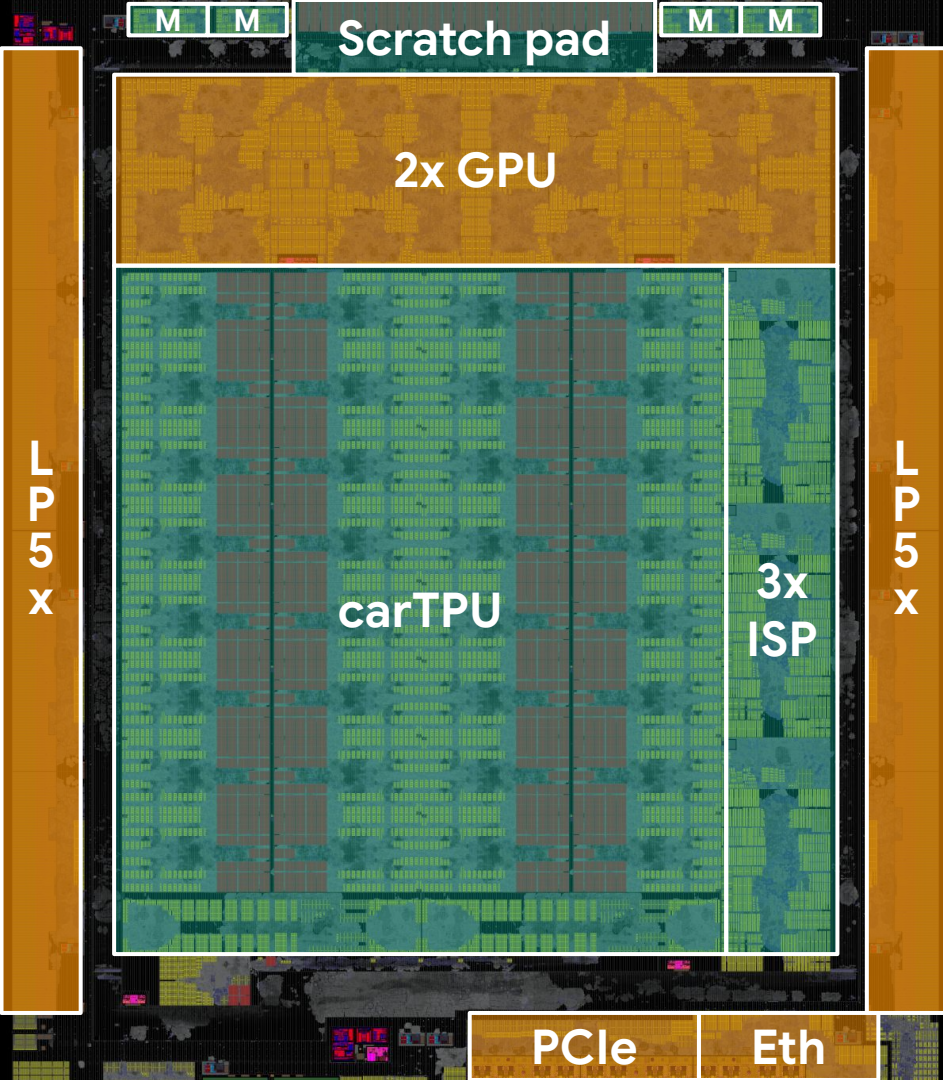
273 GB/s

Host interface

32 GB/s PCIe Gen5x8 + 25G Eth

TDP

<75 W



Major subsystems

Waymo-designed

- First generation carTPU
- ISP, codecs and MIPI ingest path
- Scratch pad memory

Proven third-party IP

- GPU
- On-chip interconnect
- Memory, PCIe, ETH Controllers
- PHYs
- Secure boot
- Peripherals

carTPU

Evolution of Waymo AI

2013

**Deep Conv
Networks**

With
primary use
in Perception

2017

Transformers

Advancing
perception
and behavior
modeling

2019

**Open
Dataset**

Most
comprehensive
dataset for
AV research

2020

**SOTA Deep
Learning**

Widely
used across
Perception,
Planning, and
Simulation

2023

**VLM /
LLMs**

Boosting
Driver
performance
with general
world
knowledge

2024

**Foundation
Model**

Driving
next-gen
AI stack:
AV expertise
+ VLM/LLMs

Key specifications

Dense tensor compute throughput

INT8 x INT8 - 160 TOPS

INT8 x INT16 - 80 TOPS

INT16 x INT16 - 40 TOPS

FP16/BF16 x BF16/FP16 - 80 TFLOPS

FP32 x FP32 - 40 TFLOPS

Sparse tensor compute throughput

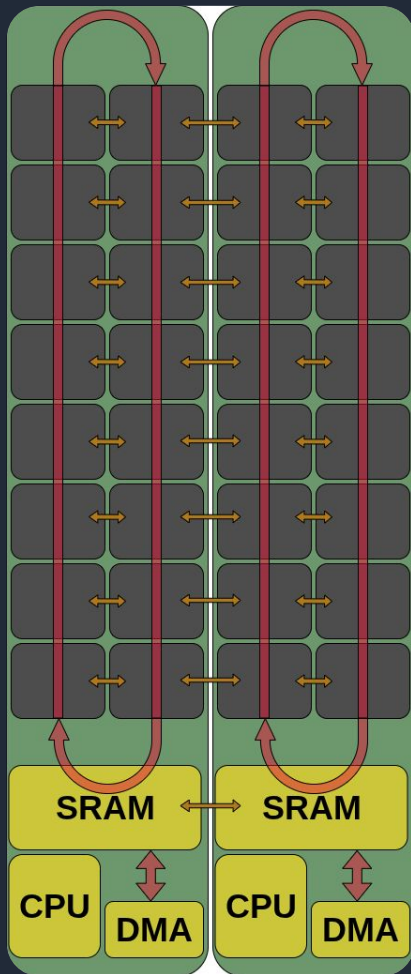
2x Dense with K:M structured sparsity

On-chip memory

64 MB SRAM + 8 MB register file

External memory BW

160 GB/s



Architecture overview

PE

- 2 MB SRAM @ 640 GB/s
- 5 TOPs (int8)
- 64B ring + 32B mesh

Control

- Dual-core RISC-V CPU w/ vector extensions
- DMA engine + 2 MB control SRAM

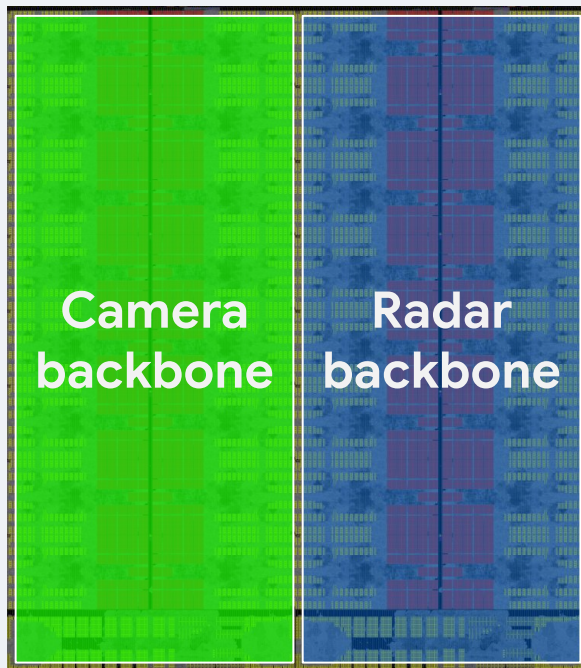
Core

- 16 PEs + Control
- 80 GB/s DRAM bandwidth
- 64B Inter-core ring + 256B (8x32B) mesh

carTPU

- 2 cores

carTPU modes

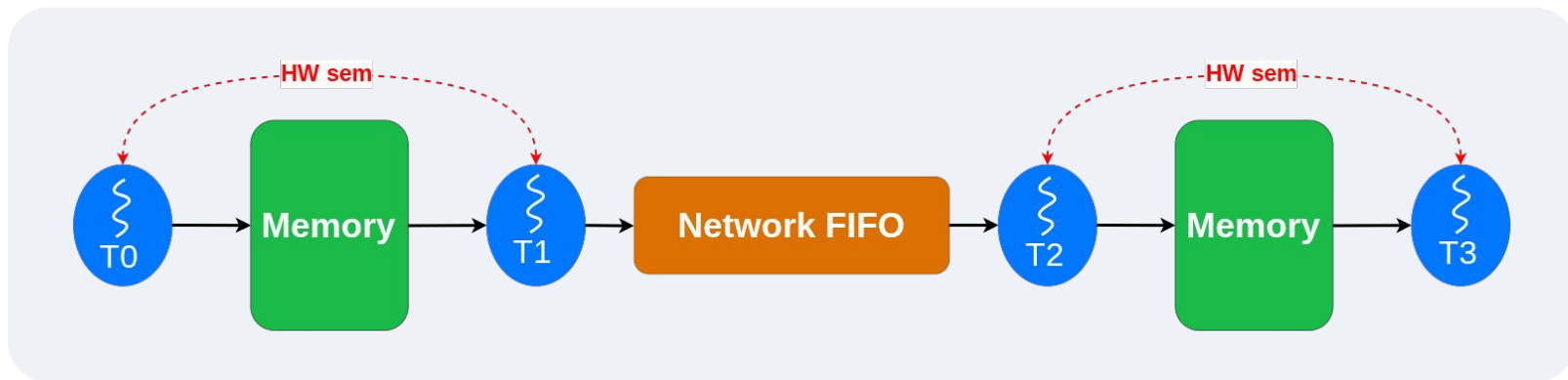


Max concurrency



Latency critical

Dataflow principles



Low HW Complexity

Static shapes and deterministic control flow:

- No caching, branches, or register allocation
- Expose full capability as spatial hardware threads (e.g. compute, DMA)

Mega-Instructions

Single thread execution over $O(1000)$ cycles:

- How hardware FSM iterates over tensor dimensions with high flexibility
- Synchronization with other threads (SPMC, MPSC)

Thread Sync

Two streaming primitives:

- FIFO: Enforces strict ordering and propagates backpressure
- Memory with semaphores: Coarse grained with relaxed ordering

PE overview

Highly banked SRAM (2 MB)

- Sized to keep activations and context resident and feed GEMVs

Ring DMA

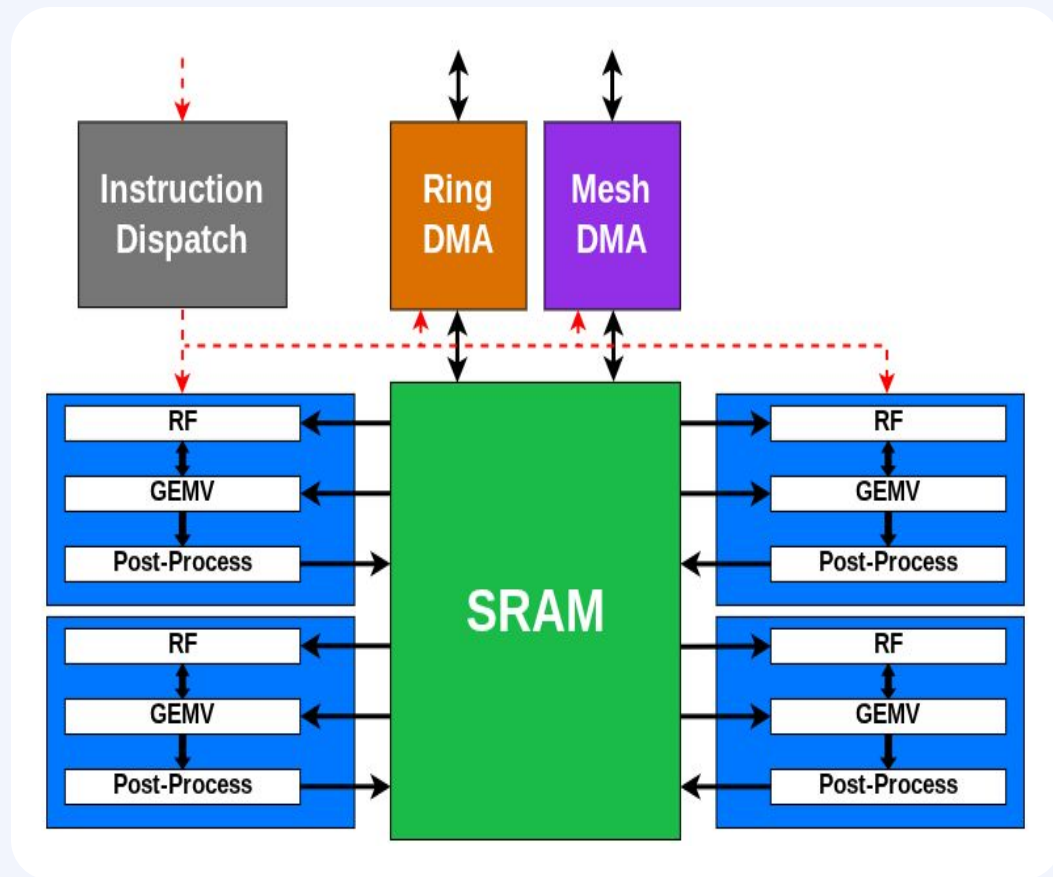
- Weight streaming
- Input load and output store

Mesh DMA

- Cross-PE collectives

4x compute slice

- 128x4* or 32x16* INT8 GEMV per cycle
- RF for reuse: GEMM, Convs
- Sparse gather/scatter fusion
- Post-process: quant-dequant, el-wise, activation functions etc.



$$*[128, 4] = [128, 4] \times [4, 1]$$

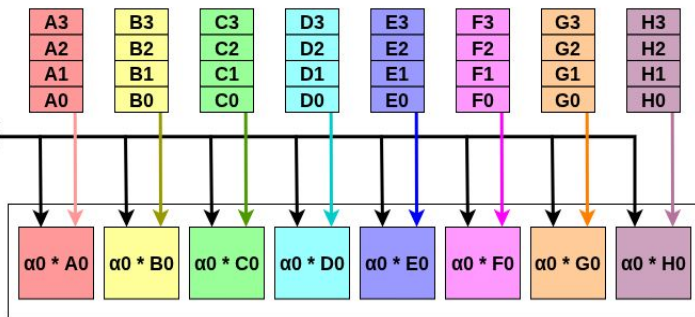
Flexible GEMV

Shape morphing for maximum spatial utilization for a variety of compute primitives

GEMV
(decode)

1 INT8

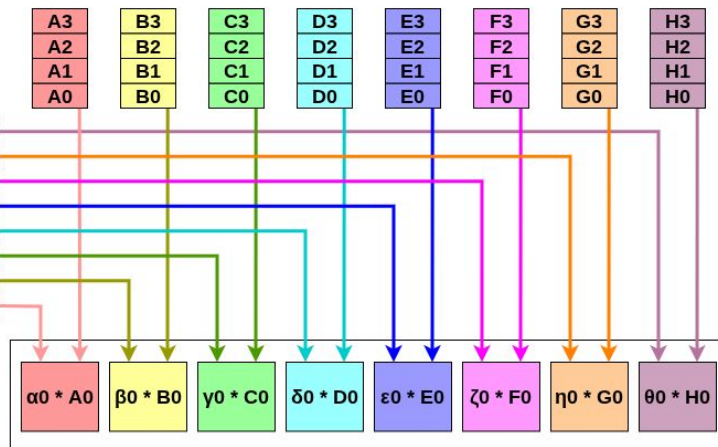
α_3 α_2 α_1 α_0



El-wise
(softmax, pooling)

1 INT8

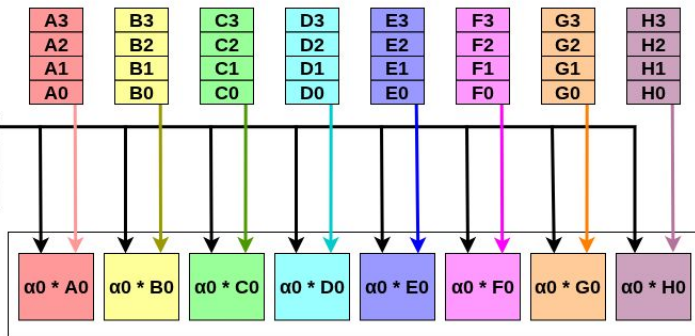
θ_3	θ_2	θ_1	θ_0
η_3	η_2	η_1	η_0
ζ_3	ζ_2	ζ_1	ζ_0
ϵ_3	ϵ_2	ϵ_1	ϵ_0
δ_3	δ_2	δ_1	δ_0
γ_3	γ_2	γ_1	γ_0
β_3	β_2	β_1	β_0
α_3	α_2	α_1	α_0



GEMM
(FFN, Convs)

4 or 16 INT8

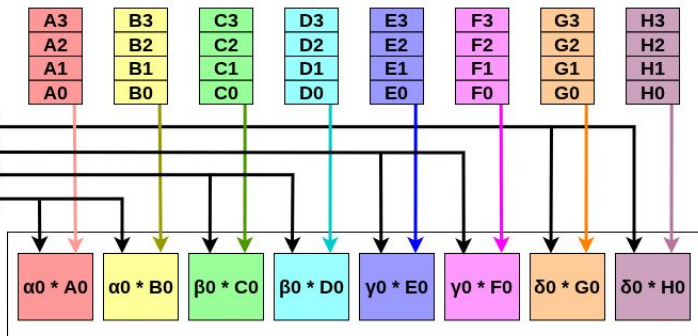
α_3	α_2	α_1	α_0
β_3	β_2	β_1	β_0
γ_3	γ_2	γ_1	γ_0
δ_3	δ_2	δ_1	δ_0

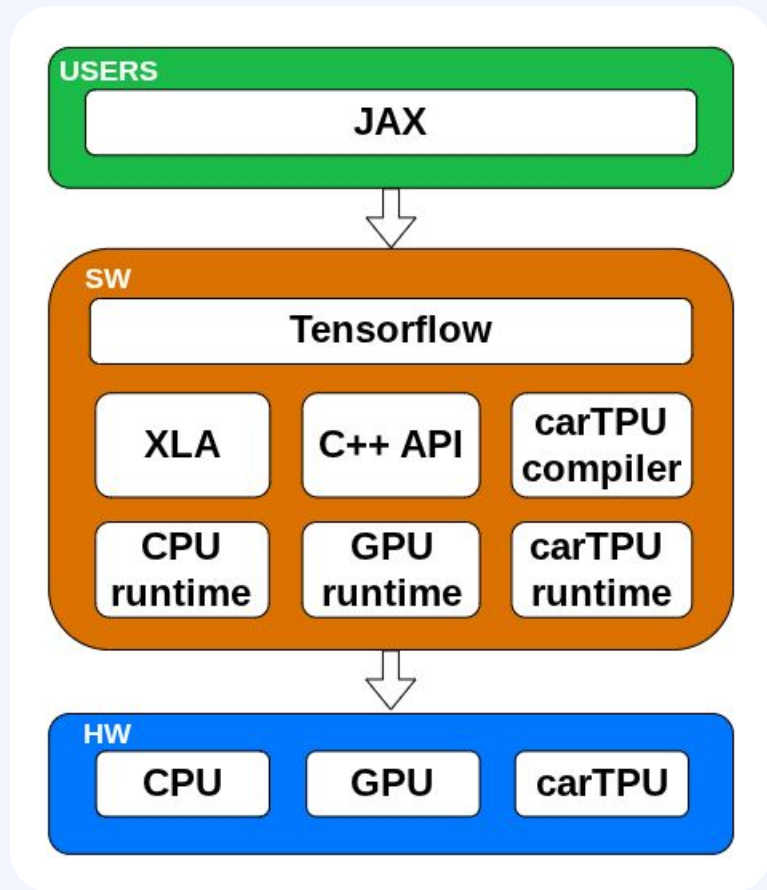


Grouped el-wise
(depth-wise conv)

1 INT8

δ_3	δ_2	δ_1	δ_0
γ_3	γ_2	γ_1	γ_0
β_3	β_2	β_1	β_0
α_3	α_2	α_1	α_0





SW stack

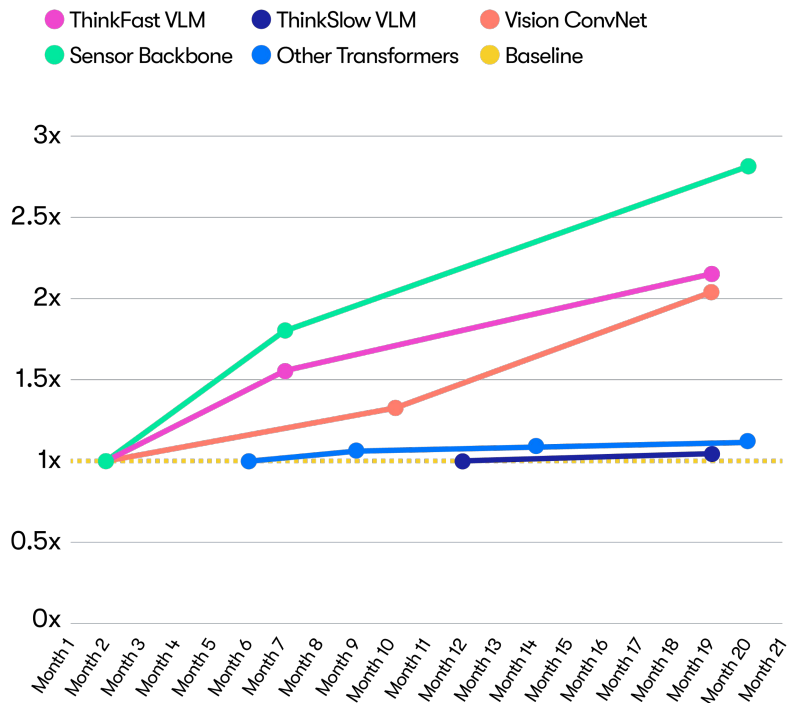
Compiler-First Philosophy

- Ahead-of-time compilation
- Flexible op placement
- carTPU SRAM-centric
- Mega-kernel for whole model

carTPU compiler challenges

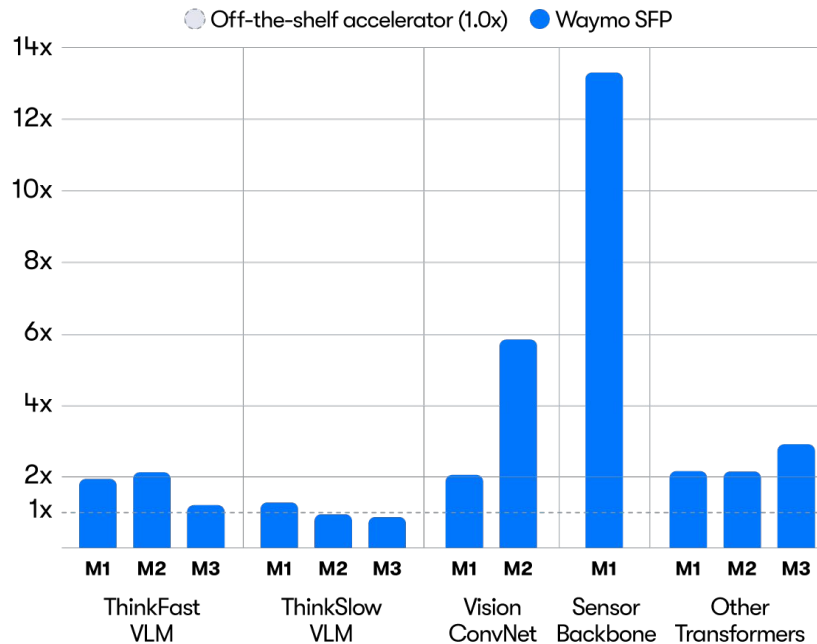
- Partitioning on distributed architecture
- Sharding to fit within SRAM capacity
- Minimizing global memory accesses
- Managing race conditions, numerical accuracy

Achieved performance: Real world performance with true co-design matters over raw TOPS



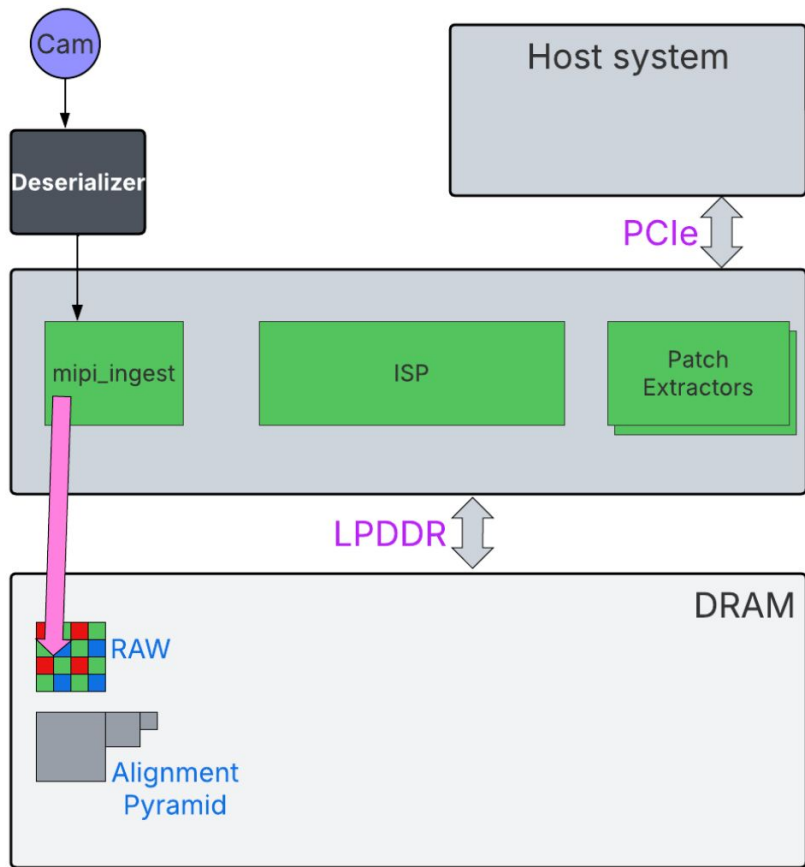
Continuous optimization: Relative speedup vs t=0 latency

$$PerformanceEfficiency_{achieved} = \frac{\frac{1}{latency}}{TOPS_{hardware}}$$



Achieved performance efficiency

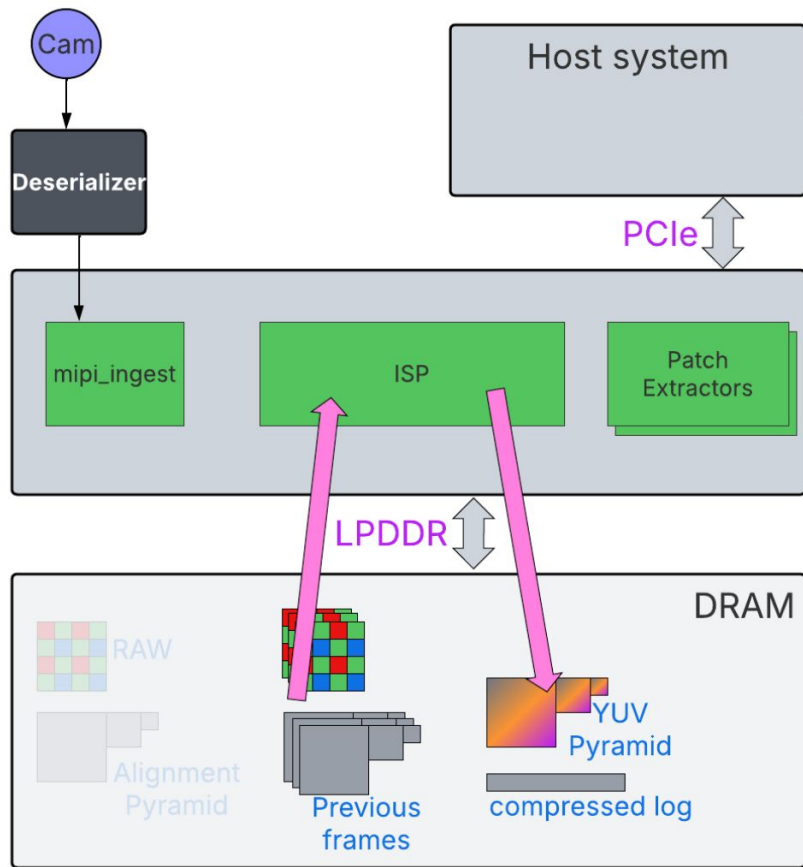
Sensor processing



Camera pipeline

MIPI ingest

- Raw sensor data to DRAM
- Grayscale alignment pyramid creation
- Sensor error handling



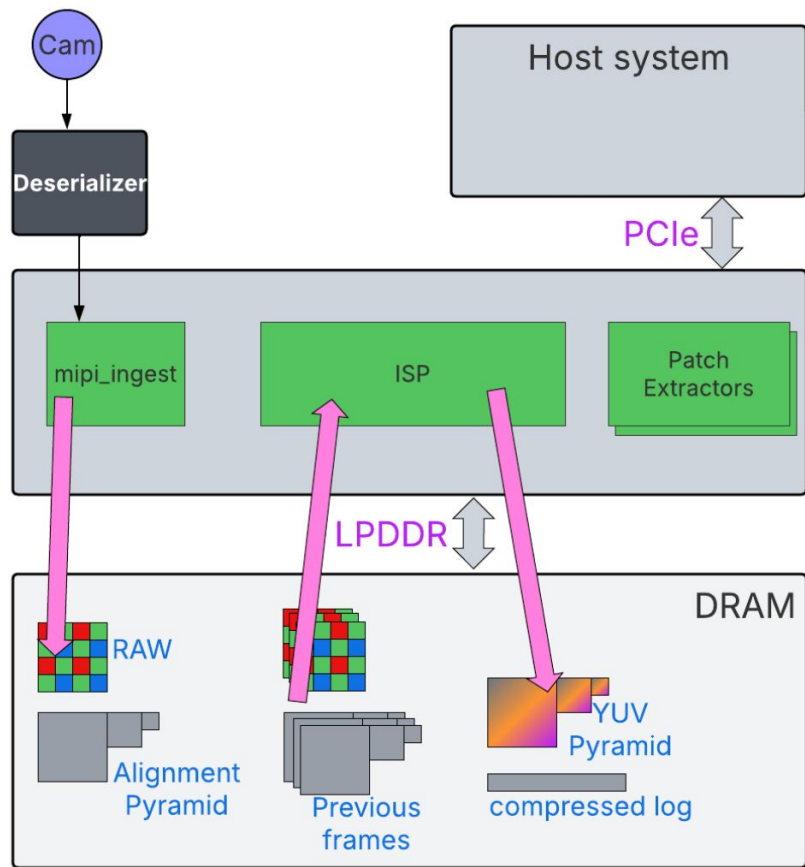
Camera pipeline

MIPI ingest

- Raw sensor data to DRAM
- Grayscale alignment pyramid creation
- Sensor error handling

HDR ISP

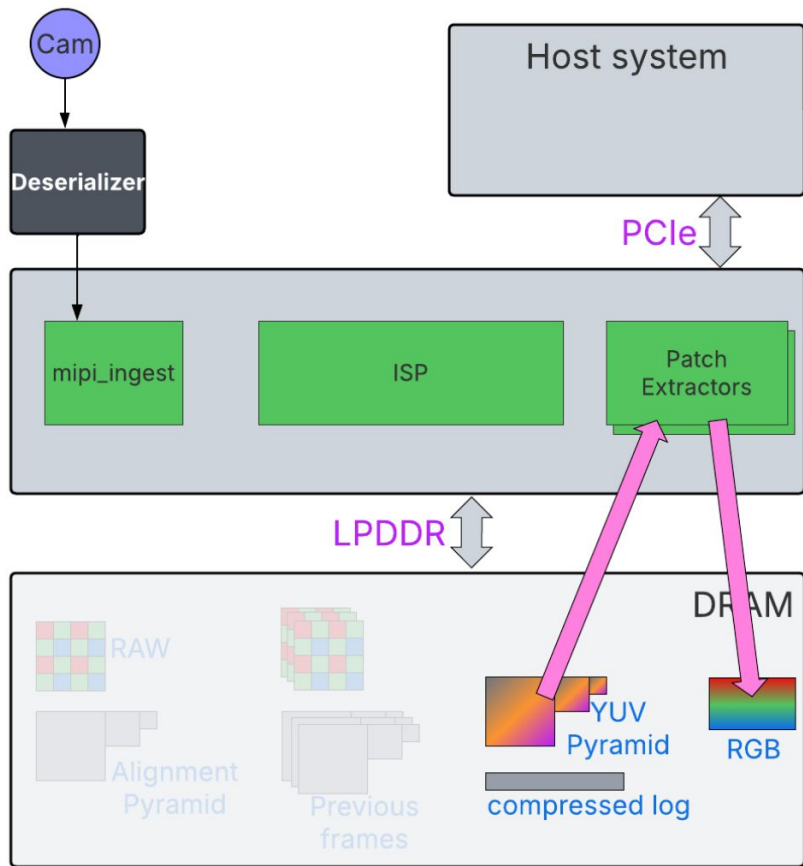
- Raw -> RGB -> YUV (temporal denoise)
- Still-frame image codec
- YUV pyramid -> DRAM



Camera pipeline

Pipelining

- MIPI Ingest outputs progress
- ISP advances as far as it can
- YUV pyramid is produced with sub-frame latency from sensor read-out
 - ~100's of image lines



Camera pipeline

MIPI ingest

- Raw sensor data to DRAM
- Grayscale alignment pyramid creation
- Sensor error handling

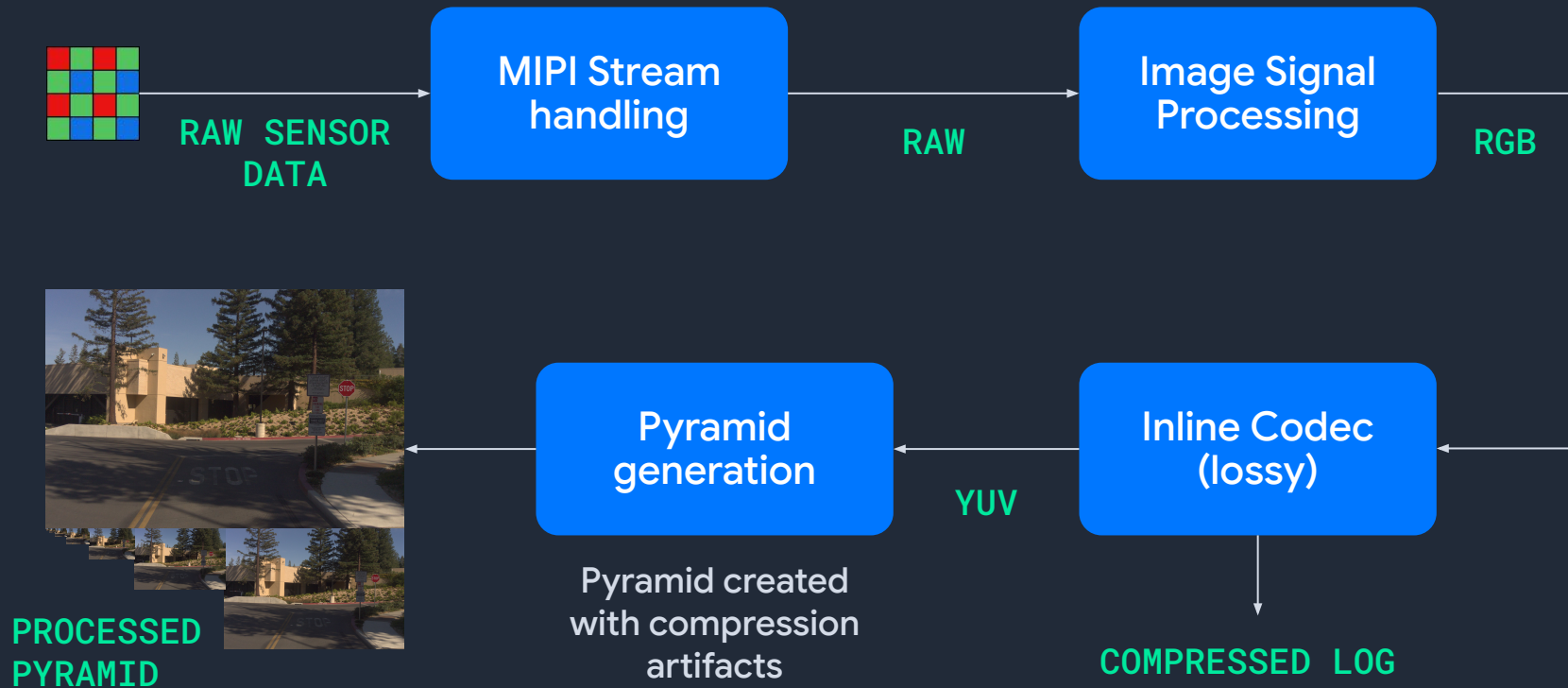
HDR ISP

- Raw -> RGB -> YUV (temporal denoise)
- Still-frame image codec
- YUV pyramid -> DRAM

Patch extraction

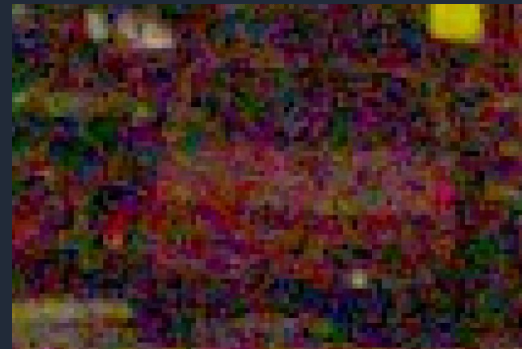
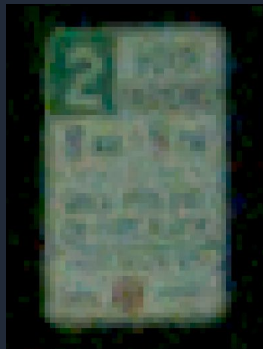
- YUV pyramid -> arbitrary scaled region-of-interest (ROI)
- Color space conversion
- Synthetic exposure of HDR

Camera pipeline

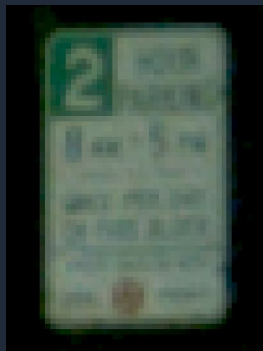


ISP: Temporal denoise

Single frame



Multi-frame
temporal
denoise



ISP: In-house demosaic

Traditional ISP demosaic can show color channel bleed



Strong zipper artifacts



In-house demosaic retains color accuracy



Edges are preserved



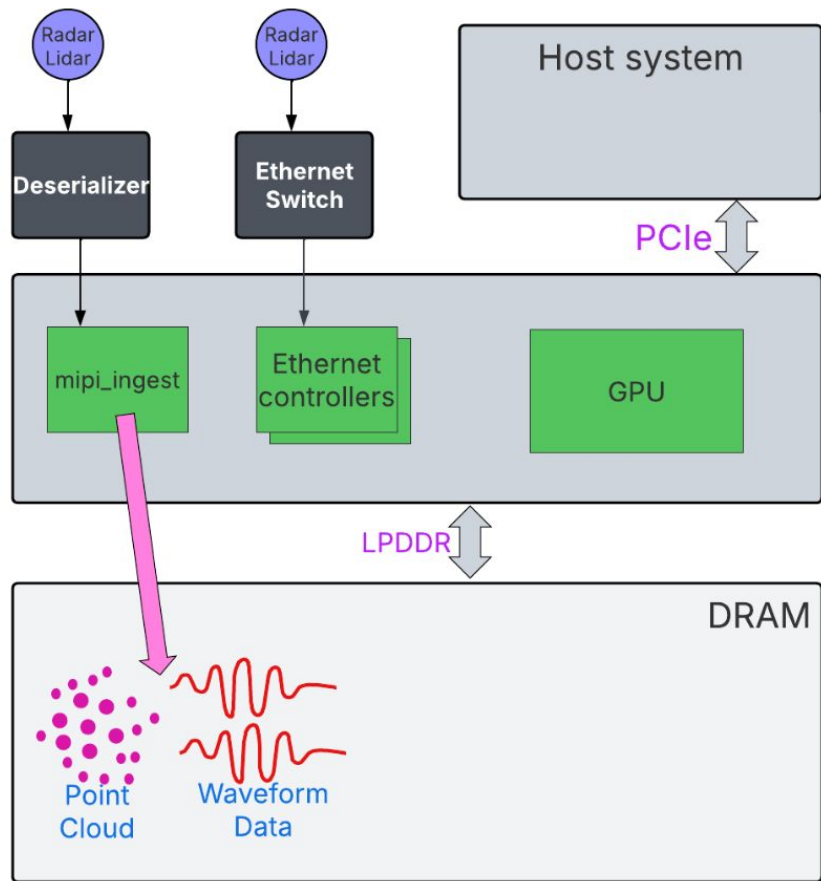
ISP: HDR sensing



Challenging HDR scenes:

- Headlight glare
- Exiting tunnels

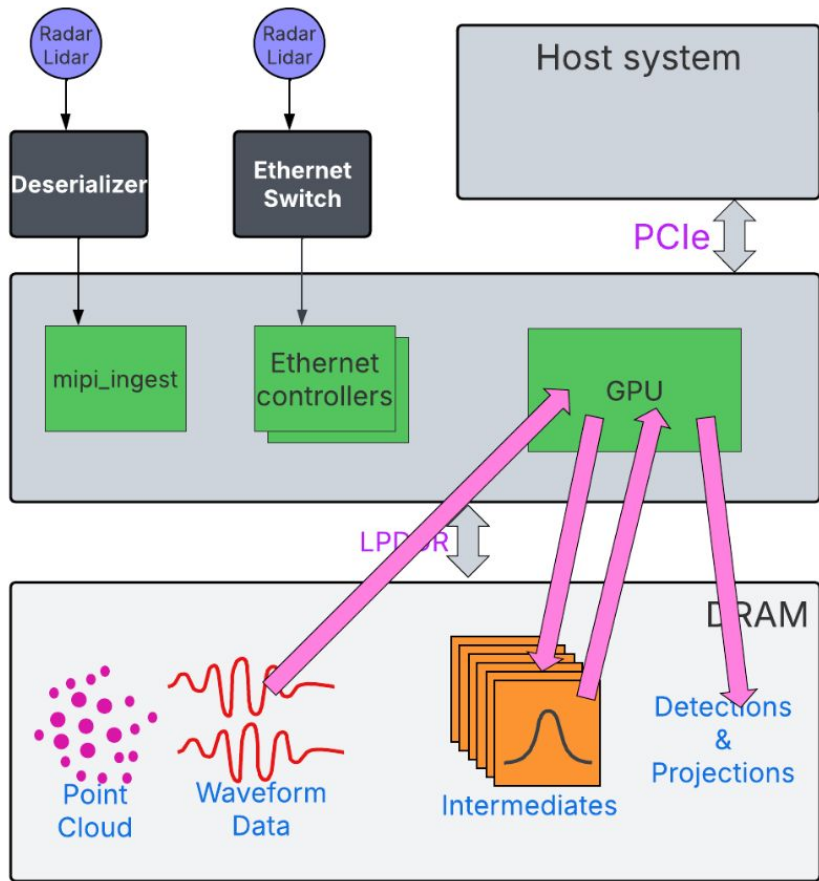




Radar & lidar pipeline

MIPI ingest

- Raw sensor data to DRAM
- Sensor error handling



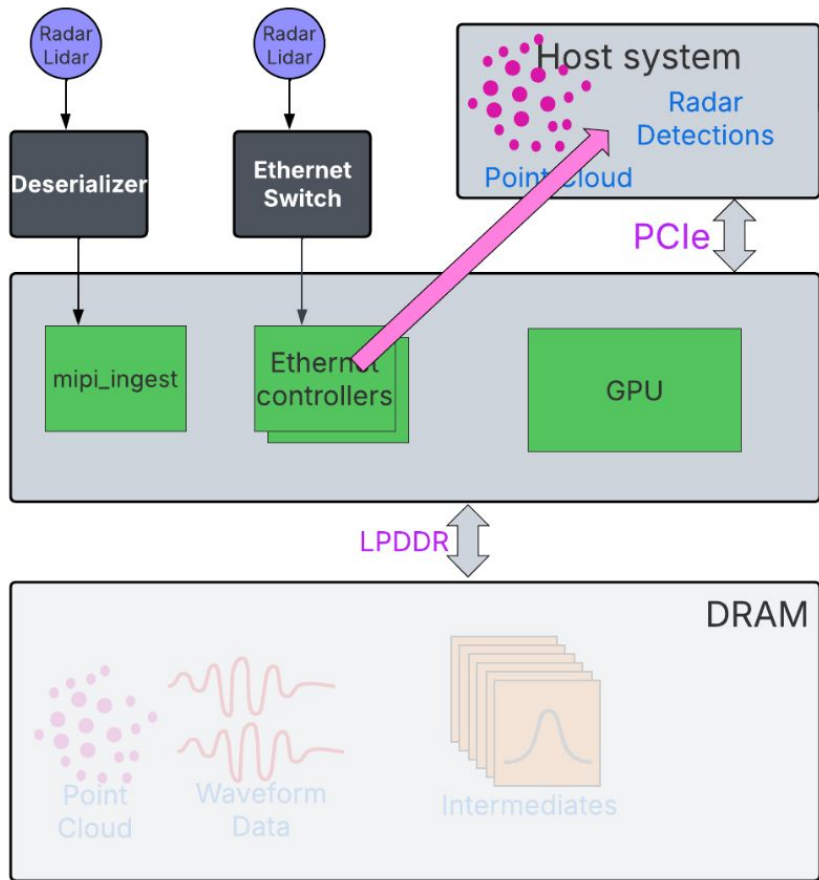
Radar & lidar pipeline

MIPI ingest

- Raw sensor data to DRAM
- Sensor error handling

GPU

- Point cloud processing
- Waveform procession & FFT cubes
- GPGPU programming models



Radar & lidar pipeline

MIPI ingest

- Raw sensor data to DRAM
- Sensor error handling

GPU

- Point cloud processing
- Waveform procession & FFT cubes
- GPGPU programming models

Ethernet

- Direct to host

Typical radar processing

Multi-dimensional FFTs

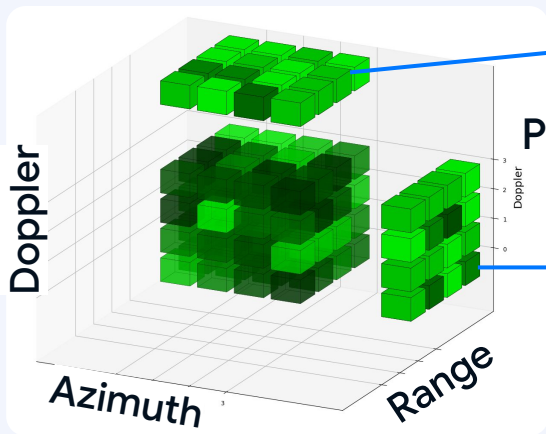
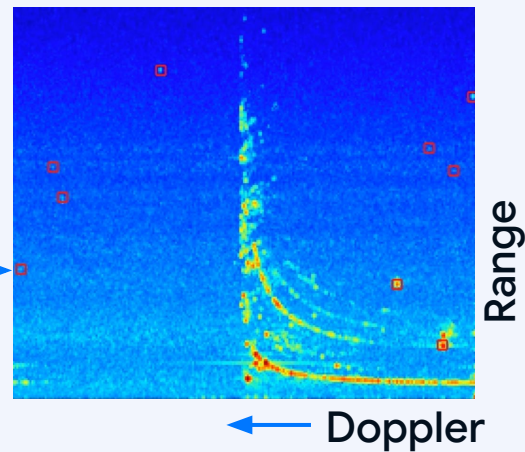
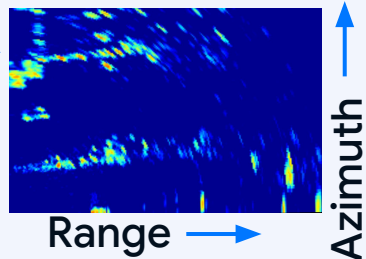
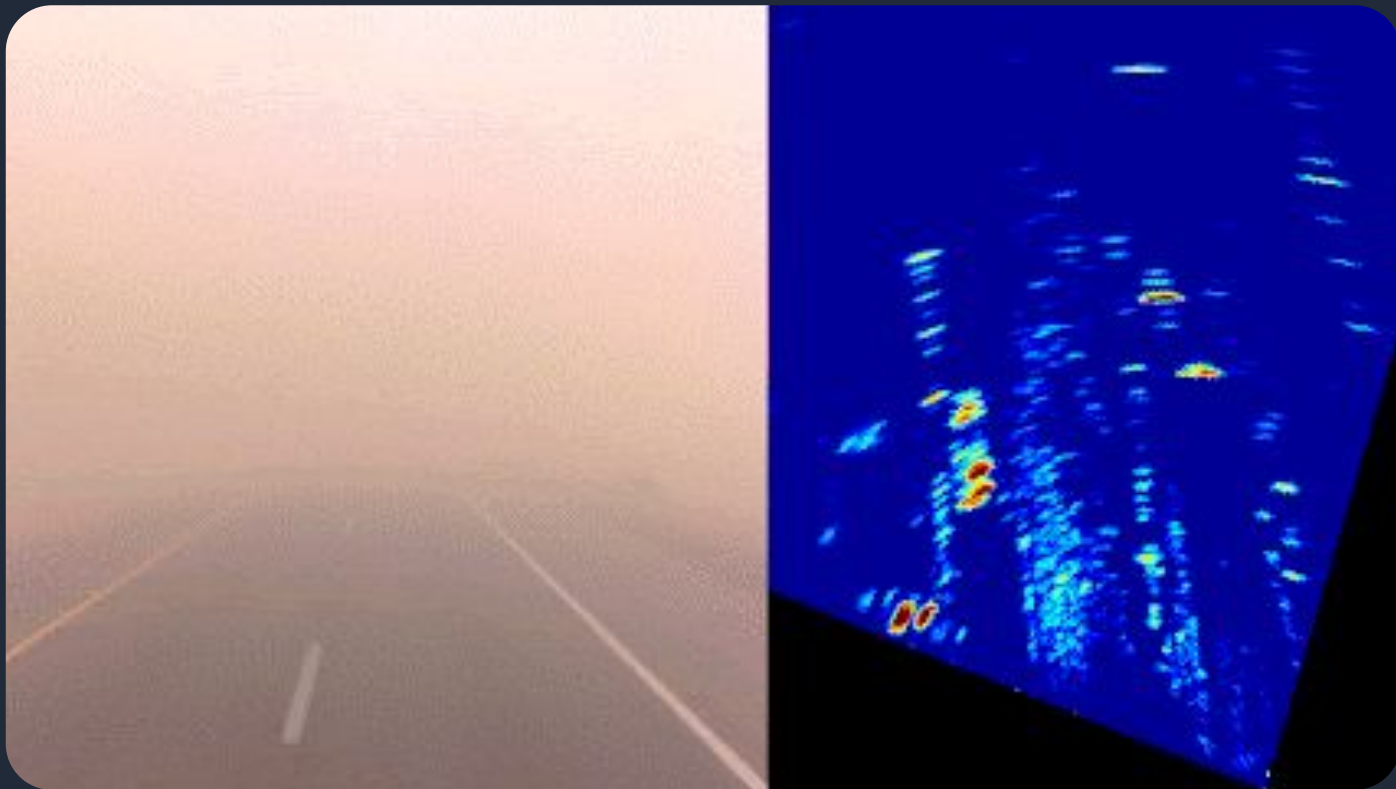


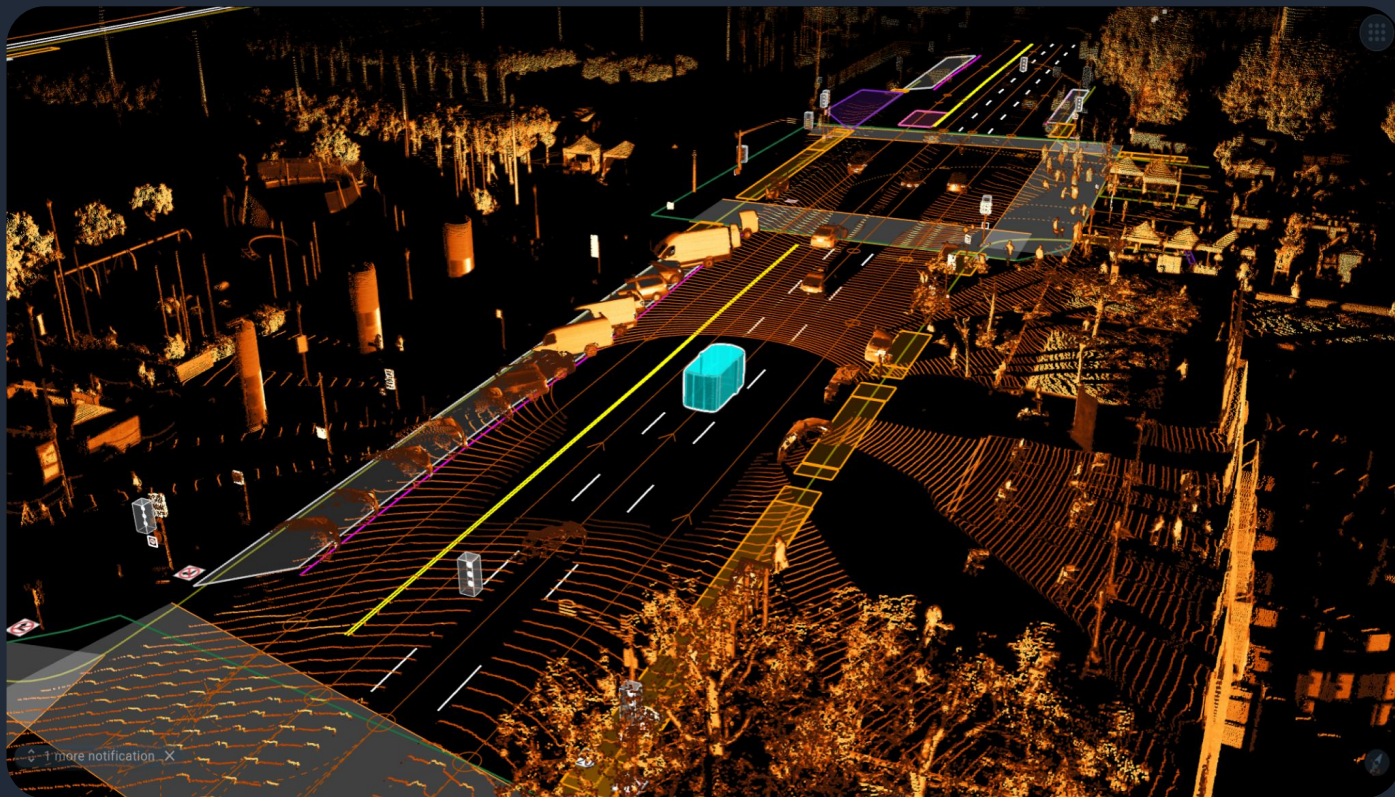
Image
Projections



Radar example image



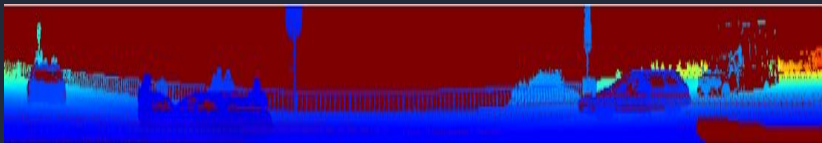
Lidar point cloud example



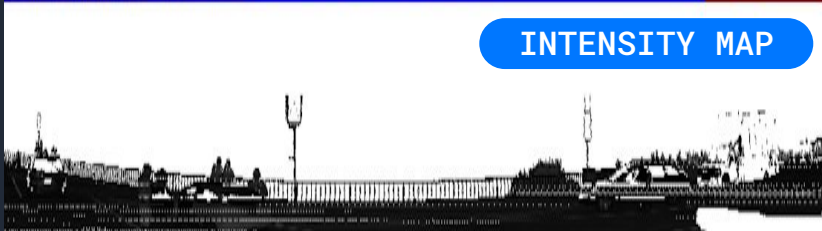
Example lidar point cloud projections

Perspective View

RANGE IMAGE



INTENSITY MAP



ELONGATION MAP

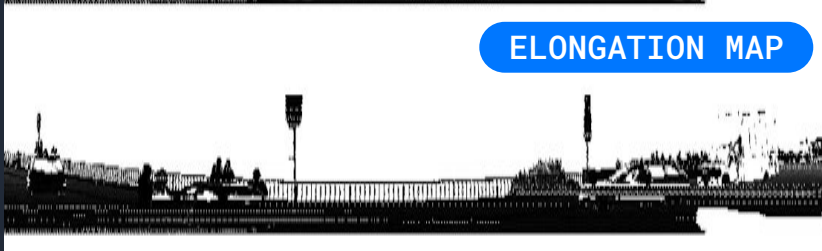


Image credit: Waymo Open Dataset

Top-Down View

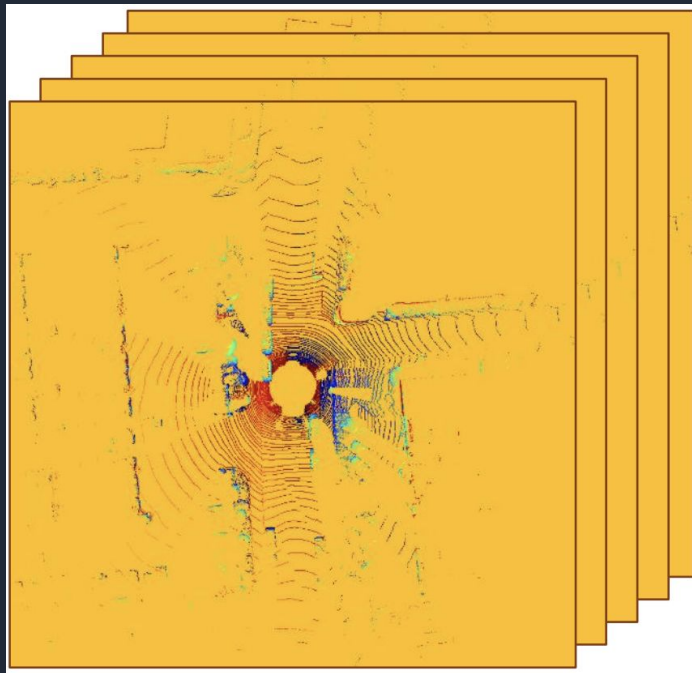


Image credit: Chen et al

WAYMO

Summary

Summary

Focus today was on sensor fusion, more to come...

First purpose-built
ASIC for autonomous
L4 driving at scale

Co-designed and
architected from
the ground up

Best-in-class
performance
efficiency for
Waymo's workloads

You can take a ride
with one in our sixth
generation vehicles



WAYMO

Huge thank you —

To our colleagues and
collaborators at Waymo who
breathed life into this chip

WAYMO

